

Bin Mixing Patterns for Robotic Pick-and-Place

Summer Internship Report - CPT, Summer 2026

Author: Ha Anh (Hanna) Le
Undergraduate, University of Pennsylvania
Robotics Application Intern, Eureka Robotics

Supervisor: Francisco Suarez
Head of Engineering, Eureka Robotics

CPT Advisor: Dr. Michael Posa
Associate Professor, University of Pennsylvania

Introduction

Problem statement

Pick-and-place tasks represent a foundational operational challenge in industrial robotics and automated logistics. While modern perception pipelines and suction or gripper end-effectors excel at isolating and picking well-oriented items, densely packed or entangled bin contents present severe operational bottlenecks. Items in cluttered bins frequently overlap, interlock, or present ungraspable surface orientations.

To mitigate these issues, active bin mixing, where the robotic arm executes a predefined trajectory through the bin contents prior to picking, serves to redistribute, untangle, and reorient items into graspable configurations. This report presents a formal technical evaluation of four candidate mixing trajectory patterns: Spiral, Concentric Circles, Zigzag (Raster Scan), and Figure Eight (Bernoulli Lemniscate).

The primary objective of this study is to evaluate the trade-offs among spatial coverage, trajectory smoothness, motion execution stability, and computational overhead across varied bin geometries. All trajectory logs and evaluation data were generated using simulated joint execution on an Aubo-i5 6-DOF industrial robot controller.

Scope of project

Due to a lack of access to robot hardware, the project was run in simulation. No observation of the physical effect of mixing on bin contents (whether parts are separated or pick quality improved) is therefore reported here. The evaluation instead characterizes the motions themselves: their smoothness, the coverage of each motion, and how long it takes to compute. Coverage serves as a proxy for pattern effectiveness, on the assumption that a motion traversing a greater proportion of the region disturbs a greater number of parts.

While “departure operations,” how the arm leaves the bin after the pick, were also included in the proposal, this was a secondary objective. Given the hardware and time constraints of the internship, this objective is deferred to future work.

Background

Placement of mixing

In a production pick-and-place cell, active mixing can be integrated at three distinct operational stages:

- **Pre-pick mixing:** executed upon detecting bin clutter above a threshold prior to initiating any pick attempts.
- **Post-failure mixing:** triggered conditionally when perception detects a failed pick or no viable grasp candidates.

- **Continuous/interleaved mixing:** integrated periodically every N pick cycles to disrupt item settling and exposure of hidden items.

This report employs the first option of pre-pick mixing, and integrates it into a repeating cycle of capture, detection, grasp selection and execution (Figure 1.1 and 1.2). A screening model is included in front of that cycle: if it reports no region offering an acceptable pick, a mixing pattern is generated for the overlap regions and executed, and the cycle restarts; otherwise, the existing pipeline runs unchanged. Details of the perception and planning stages are omitted for confidentiality.

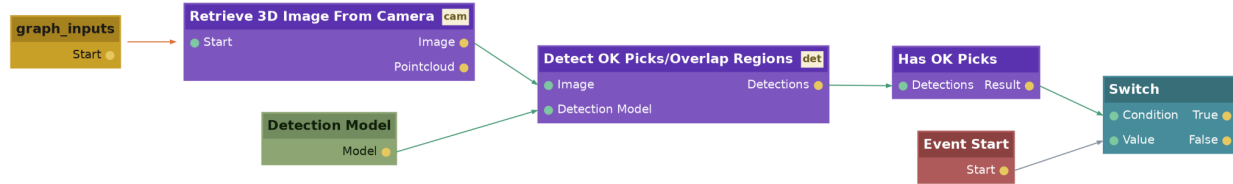


Figure 1.1. Node graph of the picking cycle where a detection model is used to detect whether there are OK picks. The two nodes “Retrieve 3D Image From Camera” and “Detect OK Picks/Overlap Regions” have the two labels “cam” and “det” respectively.

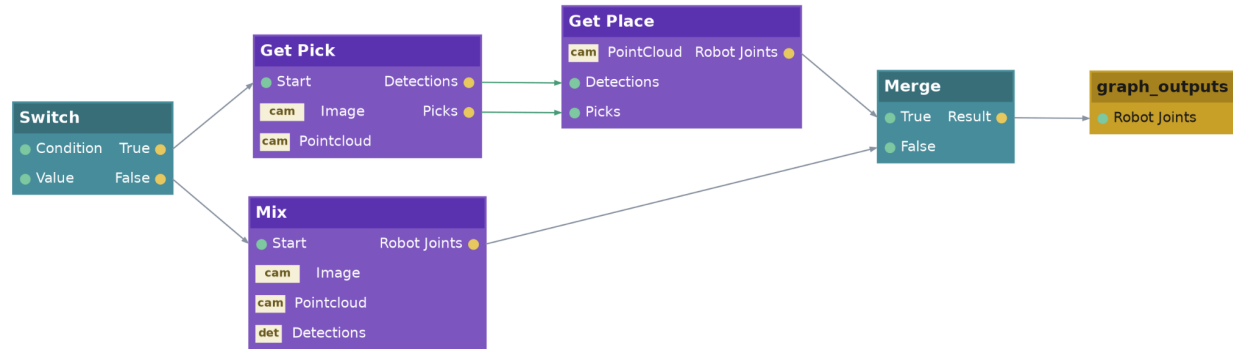


Figure 1.2. Continuation of the node graph where the result of the detection model determines whether to pick and place, or to mix. Inputs with labels (“cam” and “det”) are linked to the outputs from the respectively labelled nodes in Figure 1.1.

Related work

Separating tangled or cluttered parts has been studied mainly through learned policies that choose separating actions from observations (Zhang et al., 2023; Zhu et al., 2026), and through libraries of reusable manipulation primitives (Zhou & Jia, 2026). Drawing from these results, the bin mixing pipeline acts as a small set of reusable, parametrized primitives rather than planning each one from scratch. Both treat the problem as recovering a grasp once the system has already failed. Screening first is worth trying as a region of overlapping parts can be misread as a single part: pose estimation returns a candidate that looks acceptable but describes geometry that is not there, and the pick that follows fails or lifts the wrong thing. Mixing before that stage removes the configurations that produce such candidates, so the aim is not only to recover from a stall but to raise the average quality of later picks.

Mixing patterns

Design space

To isolate geometric effects, all patterns were evaluated using uniform speed, pitch, segment length, and minimum waypoint spacing.

Choice	Setting used	Notes
Path geometry	Four different shapes as specified below (see section: Mixing patterns > Mixing patterns)	The variable under study
Starting point	Centroid of the region flagged by the clutter detector	Starting from the bin center was planned but not implemented
Depth	Constant z (set at 0.1m) for the whole motion	Fixed at $z = 0.1$ m to accommodate test object dimensions
Repeats	One pass through the whole pattern	
Spacing parameters	Same pitch (the gap between consecutive terms), segment length (sampling density), closest distance between 2 points	Comparison is between shapes rather than between tunings

Table 1. *Specifics for each parameter in the testing space.*

Mixing pipeline

Figure 2 illustrates the mixing pipeline, detailing the workflow from overlap region detection to joint trajectory execution. The starting point is derived in a separate node graph, which takes the centroid and four corner points of the detected overlap region and returns the transformed centroid at a fixed working height, together with the box spanned by the transformed corners. Hence, the patterns are generated inside the overlapped region and not across the whole bin. Every coverage figure that follows is only coverage of this region.

Each pattern is an array of 3D waypoints at a determined constant height, returned from a Python function that takes in the starting point, the four corner points of the region, and spacing parameters. All four patterns implement this interface, allowing substitution without modification to downstream stages.

Bounding constraints are enforced during pattern generation by shrinking region limits by a safety margin. If a boundary collapses during shrinking, an error is thrown to prevent out-of-bounds waypoints. A minimum-spacing pass then drops any waypoint within 15 mm of the previous one, preventing the dense clusters that otherwise form near the center of the spiral and circles.

There is no separate stopping condition on mixing. Once a pattern has been executed, control returns to the screening model at the start of the cycle and the same logic runs again. If mixing fails to resolve bin

clutter, the cycle currently repeats indefinitely; implementing a maximum retry threshold is required for production.

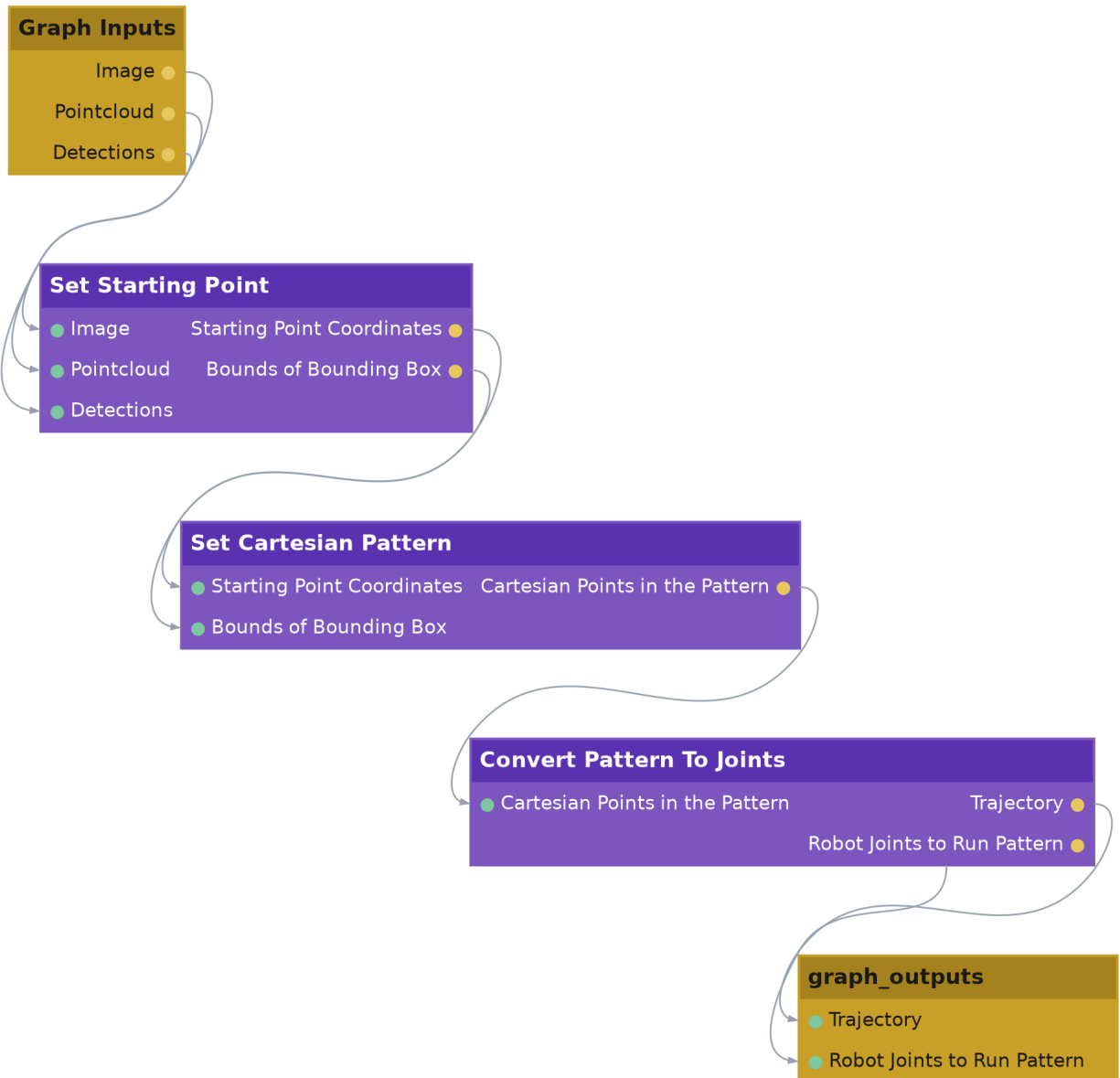


Figure 2. Node graph depicting the mixing pipeline.

Mixing patterns

There are four patterns generated and depicted below (Figure 3):

1. Spiral: grows outward from the start, the gap between turns set by the pitch and stretched to the region's aspect ratio.
2. Concentric circles: rings of increasing radius around the start, stopping at the nearest boundary.
3. Zigzag: a back-and-forth sweep along the longer axis, the only pattern that treats the region uniformly rather than radiating from a point.
4. Figure Eight: a Lissajous curve with lobes along the longer axis, crossing itself at the start so the region center is passed over twice.

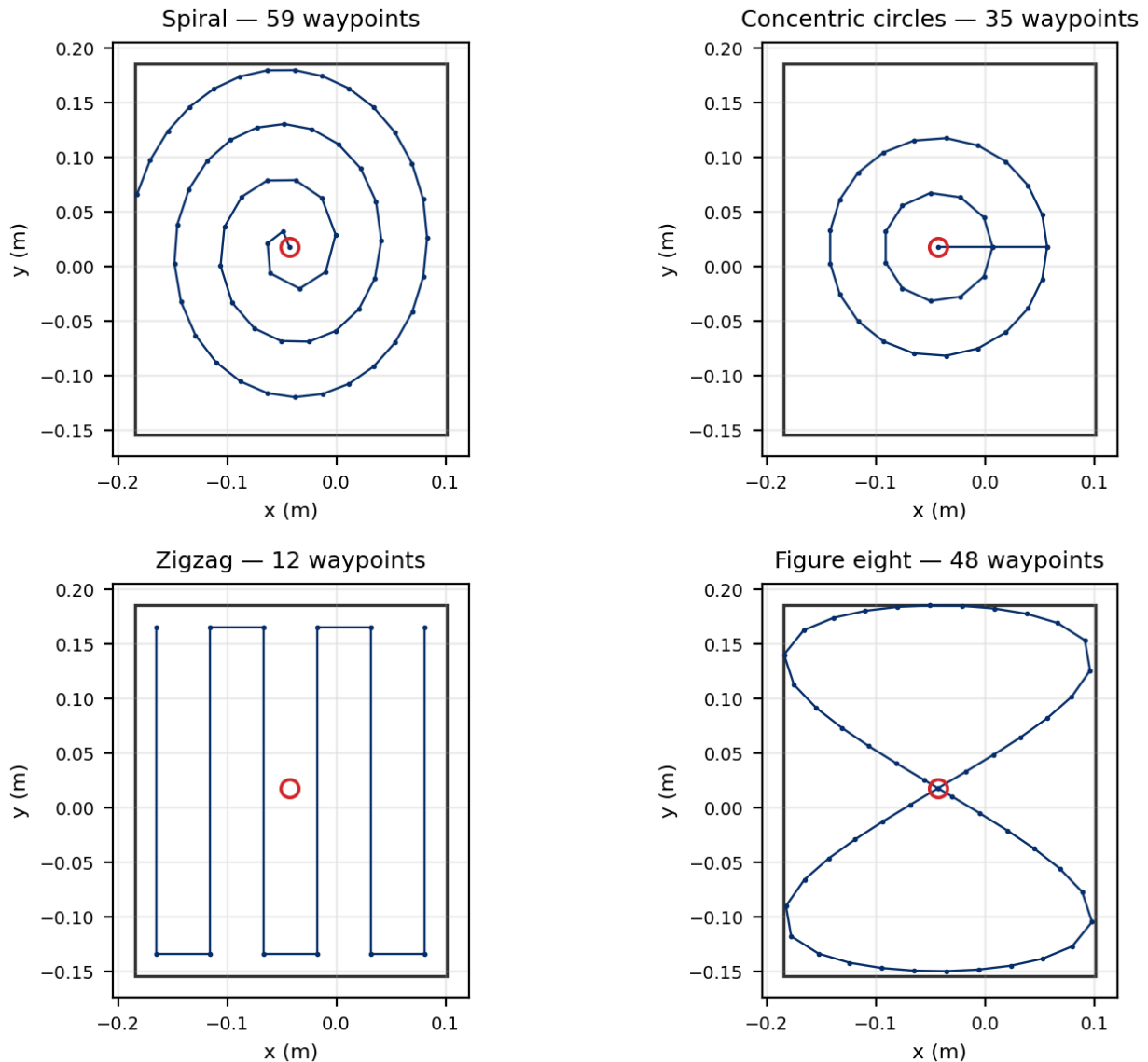


Figure 3. The four patterns generated on a real detected region, 286×340 mm. Blue dots are waypoints, the red circle is the starting point, the grey rectangle is the region boundary. Produced by `run_real_analysis.py` (Appendix A)

Evaluation method

The patterns are compared on smoothness, coverage and computation time. All patterns were generated at identical speed, pitch, segment length and minimum waypoint spacing, isolating geometry as the sole independent variable. The three measurements are implemented in `compute_smoothness.py`, `compute_coverage.py`, and `run_real_analysis.py`, which document the definitions in full. Evaluation metrics were selected based on physical path constraints and arm dynamics.

Smoothness

The smoothness is measured from the shape of the path rather than from the executed motion. Joint acceleration conflates geometric path curvature with instantaneous translational velocity. These two factors are linked by the fact that following a curve at speed v requires an acceleration proportional to v^2 and to the curvature, so on a smoothly curving path the sharpest-turning pattern imposes the greatest acceleration demand on the arm.

Path curvature is the rate at which the direction of travel changes per meter of path. Each path is resampled at an even 5 mm spacing before being measured, without which the comparison would reflect waypoint density rather than path geometry: the zigzag has 12 waypoints where the spiral has 59. Unlike continuous curves, sharp corners introduce instantaneous direction changes that create mathematical curvature discontinuities. Mean curvature is stable under the choice of resampling interval and a peak value is not, so the mean and count of waypoints turning by more than 45 degrees, and no peak, are reported together.

However, mean curvature fails to capture spatial turn distribution: while a corner and an arc through the same angle count equally, the former forces the arm to stop. Consequently, two sets of metrics are reported: mean curvature and corner count summarize path geometry, while the fastest speed profile the path admits under the arm's velocity and acceleration limits gives the load. The latter comprises the number of deceleration-acceleration cycles required and the resulting total speed variation. The corner speed limit follows from the velocity vector having to change in proportion to v when a path turns, which a controller can only do at its acceleration limit; at the limits used here, a 90-degree corner caps the speed at roughly a third of the straight-line value. Comparing curvature-based metrics with speed-profile metrics reveals significant divergence between purely geometric path smoothness and physical arm dynamics.

`compute_smoothness.py` gives both formulations and checks the curvature calculation against circles of known radius, a straight line and a right-angle corner before reporting.

Coverage

Spatial coverage is defined as the percentage of target grid cells swept by the mixing tool's effective radius. The region is divided into a grid of 5-mm cells and a cell counts as covered if its center lies within the mixing tool half-width (29.7 mm) of the path. Coverage is the proportion of cells covered, corresponding to 3,944 grid cells in the test. The path is resampled densely before this is evaluated.

Computation and execution time

Computation is timed in two stages: generating the waypoints, and converting them into a joint trajectory. Inverse kinematics conversion is the dominant computational cost. Because waypoint counts vary fivefold across patterns, conversion time depends primarily on pattern geometry rather than framework overhead. Execution time follows from the interval between waypoints, measured below.

Validity

The validity of the measurements is determined by the following conditions:

1. The measurements are stable. All measurements are exact calculations on fixed inputs, so there is no statistical uncertainty, as they are only dependent on the grid and resampling intervals chosen. Coverage varies by under one percentage point across cell sizes of 3, 5, and 10 mm, with the ranking unchanged, and mean curvature is identical to two decimal places across the same range of resampling steps (Appendix C).
2. Patterns are run at the same speed. Logged executions of the spiral and the concentric circles give 11.80 and 11.67 waypoints per second, an interval of about 85 milliseconds differing between them by 1.1%, and their joint-space step sizes agree to within 1.5%. The matching speed is the required condition for curvature calculation and comparison.
3. Curvature is able to predict what the arm experiences. From the logs of timestamped joint motion, this link can be tested. Differentiating the logged joint positions twice gives an RMS acceleration ratio between the circles and the spiral of 1.31, against a curvature ratio of 1.30, agreement to within 1%. The peak ratio is 1.55, higher and noisier, since a single sample can dominate a peak. The comparison does not require knowing the sampling interval accurately, because it cancels in a ratio; the exported timestamps resolve only to one second, which is too coarse to time individual waypoints, but sufficient to establish the rate. This requires the two runs to be at matching speed.

Limitations

There are also some limitations to the report:

1. Scope of the check does not extend to a path whose turning is concentrated at corners, where the acceleration demand is set by braking and accelerating rather than by curvature at all. This limitation is further analyzed alongside trajectory execution results.
2. As joint trajectories were only logged for the spiral and concentric circles, it remains unverified whether the zigzag pattern maintains the 85 ms waypoint interval across its longer path segments. Its curvature figure should be read as a property of its shape, not as a claim about the accelerations it produces.

Results

The results are shown for one detected overlapping region of 286×340 mm, using the 59.4 mm mixing tool, a half-width of 29.7 mm, which sets how wide a strip each pass sweeps. Parameters were held

constant across patterns, except for the safety margin; a 20 mm margin was assigned to the zigzag to prevent bin boundary collisions, whereas all other patterns used 0 mm.

Pattern	Margin (mm)	Way-points	Path (m)	Coverage (%)	Mean curvature (rad/m)	Corners	Decelerate-Accelerate cycles	Speed variation (m/s)
Spiral	0	59	1.72	84.2	12.33	4	1	1.17
Concentric circles	0	35	1.04	53.9	15.98	3	3	1.54
Zigzag	20	12	2.04	99.9	7.68	10	10	3.86
Figure Eight	0	48	1.39	61.8	7.50	4	0	0.49

Table 2. The number of waypoints, length of the path, its coverage, mean curvature, number of corners, decelerate-accelerate cycles, and speed variation of each pattern, given a fixed margin.

Pattern	Waypoints	Generation (ms)	Conversion to joints (s)	Execution (s)	Total (s)
Spiral	59	38.3	19.89	4.93	24.86
Concentric circles	35	38.9	14.67	2.89	17.60
Zigzag	12	53.3	1.81	0.94	2.80
Figure Eight	48	55.2	9.96	4.00	14.01

Table 3. The mean of two runs executed on real captures for each pattern. Execution is the measured 85 milliseconds interval between waypoints times the number of moves, so it is proportional to the waypoint count. The zigzag's segments are several times longer than the others', so its execution figure assumes the interval holds at that step size; if the arm instead slows to the common speed it would be the slowest to execute.

Coverage

The zigzag pattern achieves 99.9% spatial coverage across the target region and the spiral 84.2%, while the circles and figure eight reach 54% and 62%. The circles are limited by geometry rather than shape. The rings stop at the nearest boundary, 142 mm from the start, and are spaced 50 mm apart, so only two concentric circles fit the criteria. Hence, the pattern only spans 200 mm in a 340 mm region, regardless of how much room remains beyond it. The figure eight traverses a shorter path, which limits attainable coverage.

The tool's width also accounts for the coverage percentage. At 59.4 mm, the tool is wider than the 50 mm spacing between each zigzag pass, so adjacent passes overlap by 9.4 mm and the zigzag saturates. Even when the spacing is widened to the tool width before gaps appear, the coverage remains high: at 59 mm, the zigzag drops to five passes and 10 waypoints while still covering 97.9%, reducing waypoint count from 12 to 10 while sacrificing only 2.0 percentage points of spatial coverage. As conversion cost scales with waypoint count and dominates total preparation time, this represents the largest cycle-time reduction obtainable through parameter adjustment alone.

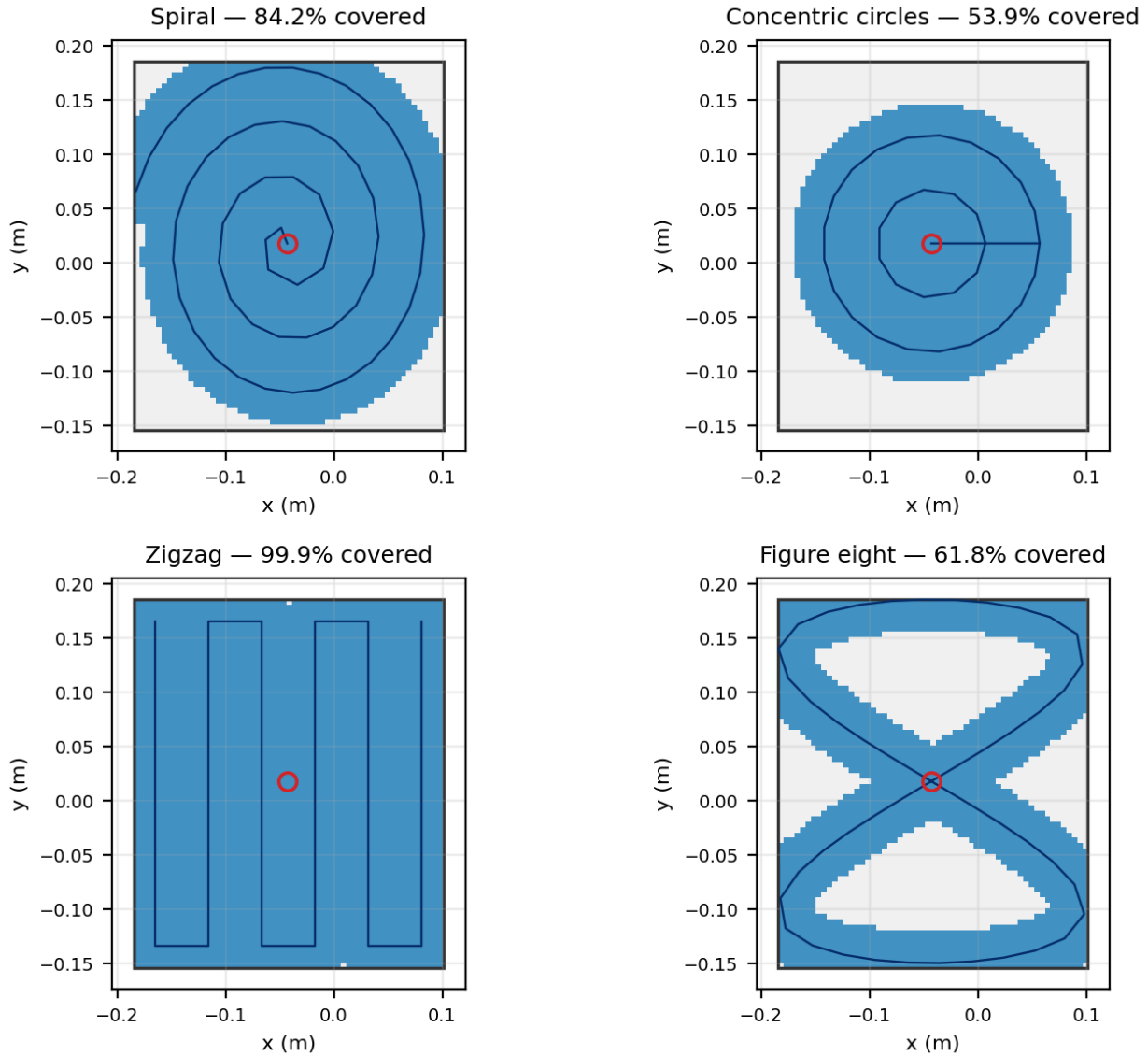


Figure 4. Region area swept by each pattern; blue is the area covered, grey is not. Produced by `run_real_analysis.py` (Appendix A)

Smoothness

Geometric mean curvature fails to reflect joint torque and load because it evaluates path geometry without accounting for sharp velocity drops at corners. By this measure the zigzag registers the lowest curvature at 7.68 rad/m, against 15.98 rad/m for the concentric circles. The measure fails because a 90-degree corner and a 90-degree arc impose substantially different loads despite subtending the same angle. While both contribute $\pi/2$, and an arc of 100 mm radius at 0.3 m/s demands 0.9 m/s^2 while a corner demands an unbounded acceleration, and in practice a stop. All of the zigzag pattern's turning points are corners, whereas the other three patterns have 65% - 77% of them on smooth arcs. Normalizing by path length compounds the effect, as extended straight sections dilute the mean.

Using joint trajectory logs from the Aubo-i5, kinematic limits were set to $v_{max} = 0.345 \text{ m/s}$ and $a_{max} = 1.9 \text{ m/s}^2$, yielding execution times consistent with dynamic constraints. The arm must slow to roughly 0.10 m/s at a 90-degree corner, so the zigzag pattern requires ten deceleration-acceleration cycles, against three for the circles and none for the figure eight patterns, and accumulates 3.86 m/s of total speed variation against 0.49 - 1.54 m/s for the others. The zigzag pattern requires the lowest inverse kinematics conversion time at 1.81 s, achieving a total preparation time of 2.80 s.

The difference between the spiral and circles with the zigzag is that the former two impose a sustained lateral load (since every point of the path is curving, the arm holds a continuous sideways acceleration throughout), whereas the latter imposes an intermittent one (long stretches without a heavy load, then ten full decelerate-accelerate cycles). Frequent deceleration cycles place significantly higher physical stress on joint actuators, a kinematic constraint uncaptured by geometric curvature metrics alone. Hence, the mean curvature is reported here as a description of path shape, and not as a measure of load on the arm, for which the decelerate-accelerate count is the better guide. For concentric rings, mean curvature is $\frac{2}{pitch \cdot (n+1)}$, predicting 13.3 rad/m for n rings versus the measured 15.98 rad/m, the difference being the radial transitions between rings. The relation shows the innermost ring radius to be bounded by the pitch irrespective of region size, so a region admitting few rings necessarily yields a sharp pattern. Empirical evaluation on physical capture data confirms this behavior: mean curvature increases by approximately 1.5 times when patterns are constrained to a region one-third the size.

Margins

Margin (mm)	Spiral	Concentric circles	Zigzag	Figure Eight
0	84.2	53.9	100.0	61.8
10	75.5	53.9	100.0	65.6
20	74.2	53.9	99.9	66.7
30	63.5	53.9	98.5	63.5
40	53.6	53.9	86.2	57.8

Table 4. Coverage (%) against the margin parameter, on the region analyzed in the Results section, at the 29.7 mm tool half-width.

The zigzag needed a margin to avoid collision with the bin. Its passes span the full short extent of the region, so at zero margin, they sit exactly on the boundary; conversion stopped partway, leaving a truncated path that executed but was not the pattern designed. This is the same property that gives the zigzag the best coverage and its indifference to the starting point position, as it is the only pattern that reaches the edge of the mixing region. The 20 mm margin resolves this, and was used for the zigzag throughout.

The margin reduces coverage by only 0.1 percentage points (from 100.0% to 99.9%). This follows from the tool geometry: the swept strip extends 29.7 mm either side of the path, so while the margin stays below that half-width, the swept strip still reaches the region boundary even though the path no longer does. Beyond the tool half-width, the two no longer overlap, leaving an untouched band at the region perimeter, and coverage falls by approximately the area of that band. At a 40-mm margin, this band is 10.3 mm wide and occupies 12.8% of the region, and the zigzag's coverage drops to 86.2%. The other patterns are considerably more sensitive over the same range, as they do not reach the boundary and shrinking the region simply shrinks them: Table 4 shows the spiral falling from 84.2% to 74.2% between 0 and 20 mm, a loss of 10.0 percentage points against the zigzag's 0.1. Applying a single margin across all patterns would therefore penalize those that derive no safety benefit from it, which is why the margin is assigned per pattern. A fixed margin should also be checked against the smallest region the detector can produce, since after subtracting the margins, the region shrinks from all four sides, removing 40 mm from each dimension and generation fails where the region is narrower than that. The margin is an exposed parameter and may be reduced accordingly.

Sensitivity to the starting point

Pattern	Minimum	Mean	Maximum	Spread (pp)
Spiral	42.3	61.7	84.5	42.2
Concentric circles	19.9	39.7	54.0	34.2
Zigzag	99.9	99.9	99.9	0.0
Figure Eight	44.1	53.3	62.8	18.7

Table 5. Coverage (%) over 25 starting points spread across the flagged region, at the 29.7 mm tool half-width and the margins given above.

The starting point is determined as the centroid of the bounding box of the overlapping region flagged by the detector, introducing positional variance across successive detection cycles. In the evaluated sample, the calculated centroid was offset by only 2.7 mm from the geometric center of the bounding box. Over 25 starting points spread across the region were chosen to generate each of the four patterns. The spiral's coverage ranges from 42.3% to 84.5% and the circles' from 19.9% to 54.0%. These correspond to spreads of 42 and 34 percentage points respectively; the figure eight varies by 18.7, and the zigzag not at all. The zigzag is invariant because its passes are determined entirely by the region bounds; the starting point selects only the entry corner. Pattern selection represents a functional trade-off between localized clutter agitation and uniform spatial coverage: where the centroid reliably localizes clutter, concentrating effort there is the purpose of screening and the variability is its cost; where the centroid is unreliable, the zigzag's insensitivity becomes an advantage.

Computation and execution time

Waypoint generation requires 38 - 55 ms, exhibiting negligible variance across geometric profiles. Converting them into a joint trajectory requires seconds, so any comparison on computation time is effectively a comparison of the conversion stage. Conversion time correlates with waypoint count but is not fully explained by it; the figure eight has more waypoints than the circles yet converts faster.

Absolute planning duration is the critical metric, as latency directly limits cycle throughput. Mixing is only invoked when the screening model finds no acceptable pick. The operational alternative comprises a complete perception and motion attempt cycle. Ten to twenty seconds remains substantial, and on these figures planning cost rather than motion quality determines production viability. This delay is offset against an unviable pick attempt and the bin disturbance it would have caused.

The zigzag pattern achieves the lowest overall preparation time (2.80 s compared to 14.01 - 24.86 s for other patterns) due to its minimal waypoint density. Its conversion is 0.151 s per waypoint against 0.207 - 0.419 s for the others, so linear segments incur lower computational trajectory-synthesis overhead than curved segments. Execution and planning scale with different pattern properties and are therefore reported separately. Execution is determined by waypoint count and the inter-waypoint interval, and remains

predictable and under five seconds throughout; planning is neither bounded nor predictable, and constitutes the dominant cost.

Conclusion and next steps

Four mixing patterns were implemented as parametrized Python functions in the EC GUI framework and compared in simulation on smoothness, coverage and computation time. On a real detected region of 286×340 mm, the zigzag covers essentially all of this region, but it imposes the highest kinematic demand on joint actuators due to ten full deceleration-acceleration cycles driven by 90-degree turns, whereas the smoothly curving patterns need none to three.

Planning cost dominates the computation and execution time: although generating a pattern takes milliseconds and executing a pattern takes under 5 s, converting it into a joint trajectory takes 10 - 20 s for three of the four patterns, a figure which determines viability in a production cycle. The zigzag's conversion was cut short by a collision with the bin, as it reached the boundary, and a 20 mm margin resolved this collision.

The patterns are divided into two kinds: the spiral and circles grow outward from the detector's centroid and so concentrate effort where clutter was flagged, at the cost of coverage that varies by 42 and 34 percentage points as the cluttered centroid moves across successive pick attempts, while the zigzag sweeps independent of the starting point.

Absent hardware access, the effect of mixing on pick success remains unevaluated. These results establish the zigzag pattern as the primary candidate for physical validation, achieving the highest coverage and the lowest total preparation time.

The next steps after this project are as follows:

1. Test on the real robot, investigate the effectiveness of each pattern, and the choice of starting point: This would establish the performance difference between start-sensitive and start-invariant patterns, and consequently, whether the overlapping regions' centroids are good starting points. This will also be a step towards creating new starting points and incorporating departure operations.
2. Reduce conversion time: The current conversion time dominates the cycle cost and is the main obstacle to deployment.
3. Scale pitch and margin with region size: Both of these parameters are currently fixed, making the spiral and circles sharper in small regions, limiting the circles to two rings, and making a fixed margin unusable on small regions.
4. Develop a limit for mixing: Because control returns to the screening model immediately following execution, an unresolvable bin could trigger an infinite mixing loop unless a retry cap is implemented.
5. Varying depth and repeats, selecting a pattern automatically from the output of the detection model, and testing other part shapes, as entanglement is dependent on part geometry.

Software

Analysis and results figures (Figures 3, 4) were produced in Python using NumPy, SciPy, and Matplotlib; the node graphs (Figures 1.1, 1.2, 2) were drawn with Graphviz.

References

Zhang, X., Domae, Y., Wan, W., & Harada, K. (2023). Learning to Dexterously Pick or Separate Tangled-Prone Objects for Industrial Bin Picking. *IEEE Robotics and Automation Letters*, 8(8), 4919–4926. doi:10.1109/lra.2023.3291271

Zhou, H., & Jia, K. (2026). BiNoMaP: Learning Category-Level Bimanual Non-Prehensile Manipulation Primitives.

Zhu, J., Tie, C., Cao, X., Wang, Y., Guo, J., Chen, Z., ... Shao, L. (2026). AdaptPNP: Integrating Prehensile and Non-Prehensile Skills for Adaptive Robotic Manipulation.

Appendix

A. Code

The scripts provided below produce the results reported above:

- `compute_coverage.py` defines coverage: the grid, the tool-radius test, why the path is resampled before the query, and the sensitivity of the result to cell size
- `compute_smoothness.py` defines curvature and the speed-profile reconstruction
- `run_real_analysis.py` produces every number, table and results figure reported above from one configuration block
- `calculate_pattern.py` contains the four pattern generators

The scripts can be accessed here: [Scripts for Bin Mixing](#)

B. Parameter values

Every pattern was generated with the following parameters. A dash means that the parameter does not apply to that pattern.

Parameter	Meaning	Spiral	Circle	Zigzag	Figure Eight
pitch	Radial/line spacing (m)	0.05	0.05	0.05	-
seg_len	Target spacing between waypoints (m)	0.03	0.03	-	0.03
min_chord	Minimum spacing kept after thinning (m)	0.015	0.015	0.015	0.015
margin	Clearance inside the bounds (m)	0	0	0.020	0
clip	Slide along wall vs. stop at the wall	False	False	-	-
aspect	x/y stretch or derived from bounds (if None)	None	-	-	-
center_out	Sweep outward from start in both directions	-	-	False	-
scale	1.0 fills the available space	-	-	-	1.0
n_dense	Internal sampling before arc-length resampling	-	-	-	4096

C. Discretization sensitivity

Coverage at the three grid cell sizes h , and mean curvature at three resampling steps Δs . Coverage varies by under one percentage point and the ranking does not change; mean curvature is identical to two decimal places.

Pattern	C, $h = 3 \text{ mm}$	C, $h = 5 \text{ mm}$	C, $h = 10 \text{ mm}$	κ , $\Delta s = 3 \text{ mm}$	κ , $\Delta s = 5 \text{ mm}$	κ , $\Delta s = 10 \text{ mm}$
Spiral	84.2	84.2	83.2	12.33	12.33	12.33
Concentric circles	54.2	53.9	53.2	15.98	15.98	15.98
Zigzag	100.0	99.9	100.0	7.68	7.68	7.68
Figure Eight	61.9	61.8	61.7	7.50	7.50	7.50